

1. Introduction

Dans le cadre du TER, j'ai étudié l'algorithme de Prim pour la génération de labyrinthes.

2. Principe théorique

L'algorithme de Prim est un algorithme de construction d'arbre couvrant minimal. Adapté aux labyrinthes, il permet de générer un labyrinthe parfait.

3. Étapes de l'algorithme

1. Choisir une cellule de départ
2. Ajouter ses murs à une liste
3. Sélectionner un mur aléatoire
4. Si la cellule voisine n'est pas visitée :
 - supprimer le mur
 - ajouter ses murs à la liste
5. Répéter jusqu'à épuisement des murs

4. Implémentation

Description de la structure de la classe Maze.

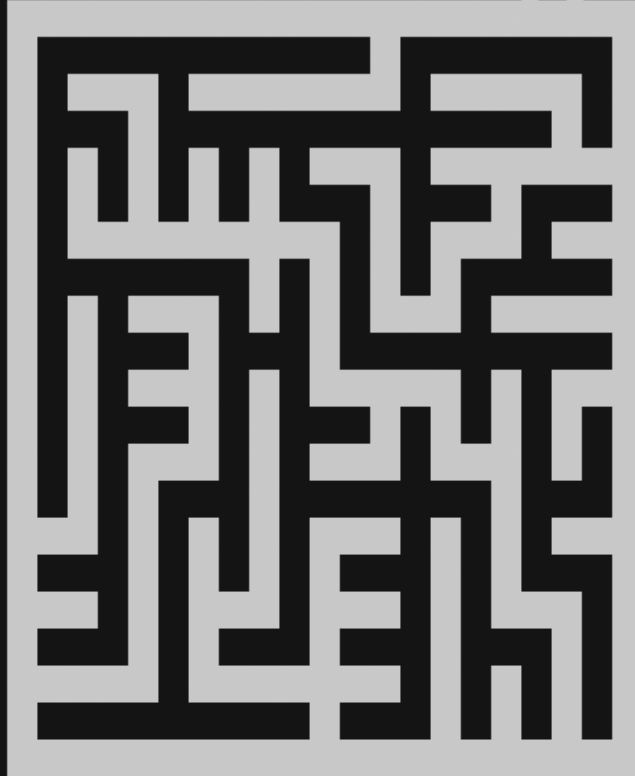
Explication des fonctions principales :

- generate_prim()
- get_walls()

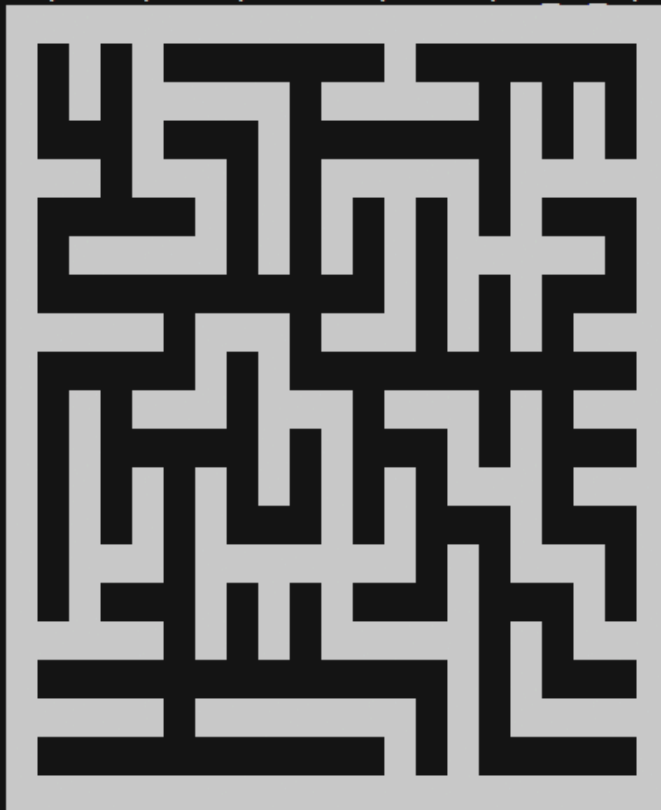
5. Tests

Tests réalisés :

```
C:\Users\nadaz\OneDrive\Bureau\TER S2 D\Proto>python maze_Prim.py
```



```
C:\Users\nadaz\OneDrive\Bureau\TER S2 D\Proto>python maze_Prim.py
```





6. Comparaison avec DFS

Prim :

- Structure plus homogène
- Expansion globale

DFS :

- Chemins plus longs
- Structure plus sinueuse

7. Conclusion

L'algorithme de Prim fonctionne correctement pour générer des labyrinthes parfaits.
Prochaine étape : ajouter des boucles.